# Python Engineering - Quick Reference

## Class 05 - Lists & Tuples: guia 'para tontos'

Referencia rapida para **week_02_data_structures -> class_05_lists_tuples**. Objetivo: poder abrir este PDF dentro de unos meses y recordar rapidamente que es cada estructura, como se indexa, como se modifica y que operaciones admite.

## 1. Lista: definicion sencilla

Una **lista** es como una caja ordenada con varios compartimentos. Cada compartimento tiene una posicion (indice) y puede contener casi cualquier objeto de Python. Las listas son **mutables**: se pueden cambiar despues de crearlas.

```
heights = [0.0, 5.0, 9.9, 14.7]
          ^     ^    ^     ^
index     0     1    2     3

heights[-1]   # ultimo elemento -> 14.7
```

## 2. Tupla: definicion sencilla

Una **tupla** agrupa varios valores en un orden fijo. Se parece a una lista, pero su estructura es **inmutable**: no puedes reemplazar, anadir o eliminar elementos de la tupla despues de crearla.

```
state = (0.1, 5.0, 49.019)
#        t    h      v

t, h, v = state   # tuple unpacking
```

**Importante:** una tupla inmutable puede contener objetos mutables (por ejemplo una lista). La tupla no cambia de referencias, pero la lista interior si puede cambiar.

## 3. Indexacion

Indexar significa acceder a un elemento indicando su posicion. Python empieza a contar desde 0.

```
values = [10, 20, 30, 40]

values[0]   # 10
values[2]   # 30
values[-1]  # 40
values[-2]  # 30
```

## 4. Indexacion anidada

Si una estructura contiene otra estructura, utilizamos un indice por cada nivel.

```
# Lista dentro de lista
matrix = [[1, 2, 3],
          [4, 5, 6]]

matrix[1]      # [4, 5, 6]
matrix[1][2]   # 6

# Lista dentro de tupla
data = ([10, 20, 30], 'sensor_A')

data[0]        # [10, 20, 30]
data[0][1]     # 20

# Tupla dentro de lista
trajectory = [(0.0, 0.0, 50.0),
              (0.1, 5.0, 49.019)]

trajectory[1]       # (0.1, 5.0, 49.019)
trajectory[1][1]    # 5.0
```

### Regla mental

`a[i][j]` significa: primero entra en `a[i]`; despues, dentro de ese resultado, entra en `[j]`.

## 5. Slicing: extraer varios elementos

```
values = [10, 20, 30, 40, 50]

values[1:4]   # [20, 30, 40]
values[:3]    # [10, 20, 30]
values[2:]    # [30, 40, 50]
values[::2]   # [10, 30, 50]
```

El extremo derecho no se incluye. `[start:stop:step]` sigue la misma filosofia que range().

## 6. Operaciones basicas con listas y tuplas

| Operacion | Ejemplo | Resultado / significado |
|---|---|---|
| Concatenar (+) | [1,2] + [3,4] | [1,2,3,4] |
| Repetir (*) | [1,2] * 3 | [1,2,1,2,1,2] |
| Pertenencia | 3 in [1,2,3] | True |
| Longitud | len(values) | numero de elementos |
| Maximo | max(values) | mayor elemento comparable |
| Minimo | min(values) | menor elemento comparable |
| Suma numerica | sum(values) | suma de elementos numericos |
| Ordenar nuevo | sorted(values) | devuelve nueva lista ordenada |

**Ojo:** multiplicar una lista por 3 no multiplica numericamente sus elementos; repite la lista. Para multiplicacion elemento a elemento utilizaremos NumPy mas adelante.

## 7. Metodos principales de listas

| Metodo | Que hace | Ejemplo |
|---|---|---|
| .append(x) | anade un elemento al final | a.append(5) |
| .extend(it) | anade varios elementos | a.extend([5,6]) |
| .insert(i,x) | inserta en una posicion | a.insert(1,99) |
| .remove(x) | elimina la primera aparicion | a.remove(99) |
| .pop(i) | extrae y elimina un elemento | a.pop(-1) |
| .clear() | vacia la lista | a.clear() |
| .index(x) | indice de la primera aparicion | a.index(5) |
| .count(x) | numero de apariciones | a.count(5) |
| .sort() | ordena la propia lista | a.sort() |
| .reverse() | invierte la propia lista | a.reverse() |
| .copy() | copia superficial nueva | b = a.copy() |

## 8. Metodos principales de tuplas

Las tuplas tienen muchos menos metodos porque no se pueden modificar.

| Metodo / funcion | Uso |
|---|---|
| .count(x) | cuantas veces aparece x |
| .index(x) | indice de la primera aparicion de x |
| len(tuple) | numero de elementos |
| max(tuple) | maximo si los elementos son comparables |
| min(tuple) | minimo si los elementos son comparables |

## 9. Modificar: lista si, tupla no

```
a = [10, 20, 30]
a[1] = 99       # OK -> [10, 99, 30]

b = (10, 20, 30)
b[1] = 99       # ERROR
```

## 10. Mutabilidad y referencias

```
a = [1, 2]
b = a

b.append(3)

print(a)  # [1, 2, 3]
print(b)  # [1, 2, 3]
```

a y b apuntan al mismo objeto. Si quieres una lista independiente: `b = a.copy()`.

## 11. Patrones utiles de ingenieria

```
# Serie temporal por variable
times = []
heights = []

times.append(t)
heights.append(h)

# Serie temporal por estado
trajectory = []
trajectory.append((t, h, v))

# Altura maxima de la trayectoria
max_h = max(state[1] for state in trajectory)
```

## 12. Que elegir?

| Necesidad | Estructura |
|---|---|
| Coleccion que va a cambiar | list |
| Estado fijo con varios campos ordenados | tuple |
| Historial por variable | varias listas |
| Historial por epoca / estado | lista de tuplas |

## Mini cheat sheet

```
LIST
[]                  create
a[i]                indexing
a[i:j]              slicing
a.append(x)         add one
a.extend(xs)        add many
a[-1]               last item
len(a), max(a)      useful functions
```

```
TUPLE
()                  create
t[i]                indexing
a, b, c = t         unpack
immutable structure

NESTED
x[i][j]             level 1, then level 2
```